

Remove This Dependency

2 October 2026 · Working document for the team · DI-Bench extension and refactoring-quality metric, combined

The idea in one paragraph

A program's real dependencies are a fact about its code, not about its manifest file. Refactoring changes the code, and a 2024 study of PyPI found that refactoring is the single largest reason manifests drift away from that fact. Dependency inference, the DI-Bench task, tries to recover that fact from the code.

So the two projects are two sides of one rule. A behaviour-preserving refactoring must leave the real dependency set unchanged, and a correct inference must reproduce it. Today's benchmarks test each side with the other frozen, and both reward passing tests, which cannot see most dependency mistakes.

The paper builds the one task where both sides meet: remove a dependency that is actually used, by rewriting the code that uses it, without changing behaviour. Then it measures how often agents cheat on that task, shows why test-based scoring cannot tell, and gives an oracle that can.

The two projects

One project is an adversary, the other is an oracle. Neither can judge the task in this document on its own.

	P1: refactoring quality metric	P2: DI-Bench extension
What it is	A learned score over a before-and-after pair of a refactoring. It combines bug-suspiciousness, structural complexity, readability and an anti-bloat penalty.	A benchmark harness. Mask a repo's dependency declarations, ask a model to restore them, rebuild in a container, run the CI tests.
What it is good at	Knowing how a change can look good and be bad. P1 built fake refactorings that game naive metrics and tested whether the score is fooled.	Knowing the ground truth. A dependency is either declared or not, the build either passes or fails. 387 model patches already exist.
What it cannot see	Dependencies, manifests, execution. It judges code text only.	Code quality, intent, cheating. A patch that passes CI is scored as correct no matter how it got there.
Where it lives	Java, 10 Apache repos, RefactoringMiner-mined commits.	Python, JavaScript, C#, Rust. Axon parses only Python, JavaScript and TypeScript.

Python is where the two can meet. RefactoringMiner now parses Python, the DI-Bench Python subset is the strongest, and every static dependency tool is Python-first. Keep Java for P1's standalone result.

The hinge: refactoring is where dependencies rot

In PyPI, more than half of declared dependencies are bloated, and the biggest single cause is refactoring that removed the last use of a library but not its declaration ([Drosos et al., FSE 2024](#)).

Cause of a bloated dependency	Share of cases
Mistake or omission during refactoring	34%
Transitive dependency listed unnecessarily	30%

Cause of a bloated dependency	Share of cases
Added and never used	16%

This is not new behaviour. [McIntosh et al., ICSE 2011](#) found that up to a quarter of source changes and almost half of test changes need a matching build-file change. Code and manifest co-evolve, and a refactoring that changes which libraries the code touches must change the manifest too.

Why this links the projects: a refactoring changes code but must not change what the code needs. So the real dependency set is an invariant of refactoring. Dependency inference tries to read that same set off the code. P1 judges the code change, P2 verifies the dependency set, and the Drosos finding says the two drift apart exactly when a refactoring happens.

Agents now refactor at scale. [A 2026 study](#) of 117,062 agent-authored dependency changes found agents pick vulnerable versions more often than humans, but it never checked whether their manifests match their imports. Nobody has.

Why passing tests is not enough

DI-Bench's headline score is execution pass rate, EPR: apply the model's manifest, build, run the CI tests, count a pass. EPR can only notice a missing dependency if some test actually imports it, directly or through project code. A dependency no test reaches is invisible to EPR.

A concrete example. A repo declares `requests` and `tqdm`. The CI tests exercise the HTTP client, so `requests` is reached. The progress bar runs only in the command-line entry point, which no test calls. A model that omits `tqdm` passes CI and scores a full pass, even though the package will crash for the first real user.

The same blindness runs the other way. A model that declares ten extra packages that happen to install cleanly also passes CI. EPR cannot tell a minimal manifest from a bloated one. DI-Bench's authors concede that "test coverage for each repository may not be exhaustive" and stop there.

This is a known weakness of tests as an oracle, not a quirk of DI-Bench. [Hejderup and Gousios](#) seeded faults into library call sites in Java projects and found test suites missed most of them. [TimeMachine-bench](#), a 2026 dependency-upgrade benchmark, reports agents producing "spurious solutions that exploit low test coverage". And SWE Atlas found a 15 to 40 point gap on refactoring tasks between passing behaviour checks and meeting maintainability rubrics.

The fix in this paper is to measure the blind spot rather than assume it away. Delete each ground-truth dependency, re-run CI, and record which deletions still pass. That fraction is the dependency mutation score of the test suite, and it bounds what EPR can ever detect.

The task: remove a dependency that is actually used

The instance is a repository, a target package that its code imports, and the instruction: make this repository no longer need that package, with behaviour unchanged. The agent must rewrite the code that uses the package, delete the declaration, and leave the tests passing.

This is the hardest point where refactoring and dependency inference meet. Removing an unused declaration is a solved problem: PyTrim and DepClean do it with static analysis and match human removals almost perfectly. Migrating from library A to library B is covered by PyMigBench and MigrationBench. Removing a library that is in use, by writing the code yourself, is neither, and it is what developers actually do when a dependency is abandoned, vulnerable, or too heavy for what it provides.

Ground truth exists. [He et al.](#) mined over 19,000 Java projects and found thousands with at least one library removal. The same mining works for Python: a commit that deletes a line from the manifest and edits a source file that imported it.

The benchmark lives in Python, on the DI-Bench repositories already in hand. Choose targets with a narrow footprint, a package imported in one or two source files, so the rewrite is bounded. The first numbers below show that pool is large.

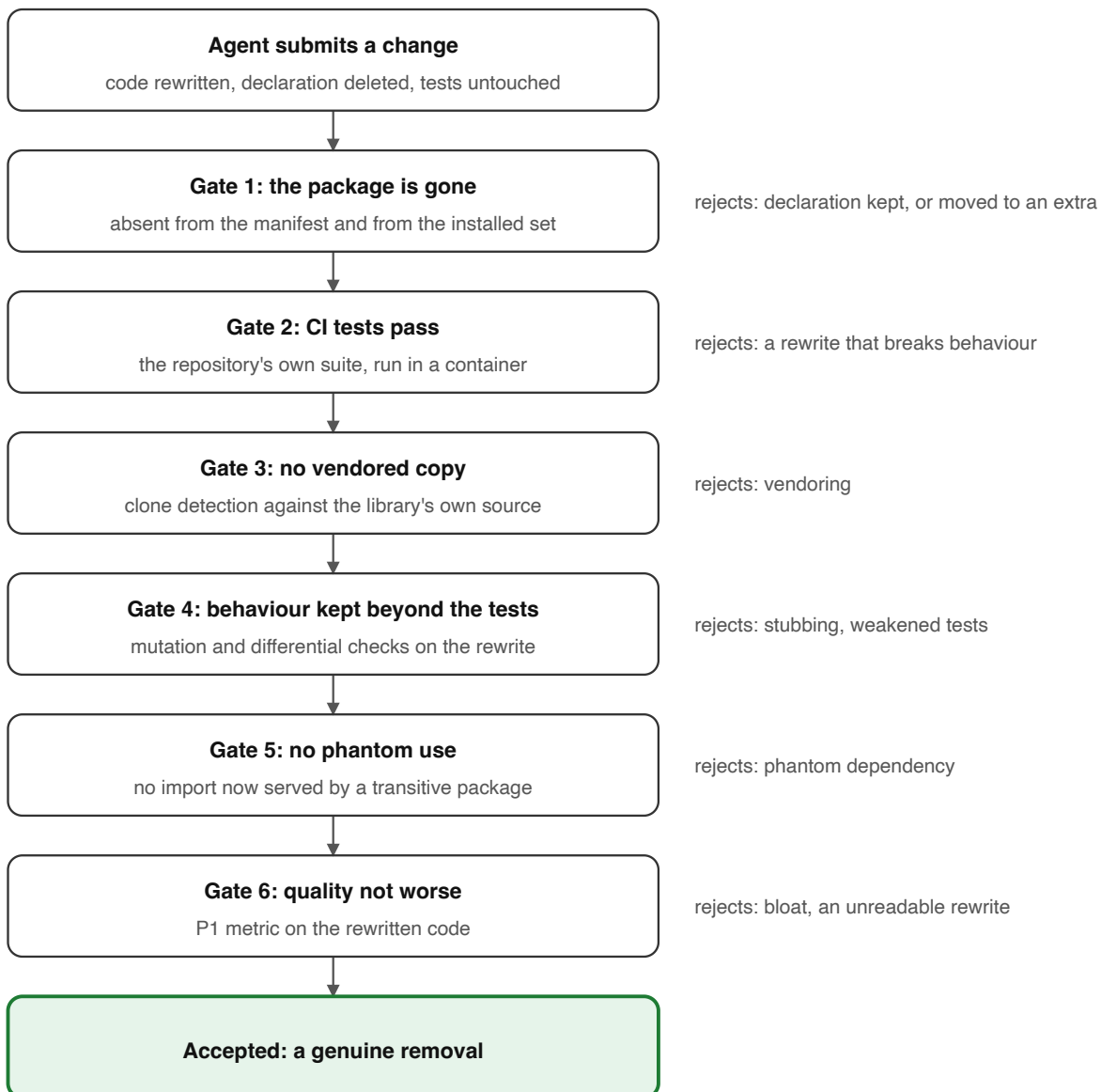
How big the benchmark can be: instances are repository and target-dependency pairs, not repositories. From the executed data on the 51 gold-passing Python repos, 139 declared dependencies are imported by one or two source files, 114 of those are reachable from tests, and 62 are in the strongest tier where CI demonstrably fails when the package is deleted, across 38 repos. A harder tier of 19 has a three to five file footprint. DI-Bench's large subset adds 50 more Python repos not yet run, and mined removal commits add instances with human ground truth. A 50-instance set is the agent pilot; the benchmark itself should target 200 to 300 instances across tiers.

How an agent can cheat, and how each cheat is caught

Every cheat below passes CI. That is the point: a test-only oracle accepts all of them, so the benchmark needs more gates than tests.

Cheat	What it looks like	Why CI accepts it
Vendoring	The library's source is copied into the repo, so the import still resolves	Behaviour is identical; the package is only hidden in the tree
Stubbing	The call is replaced by a function that returns a plausible constant or does nothing	No test exercised that path, or the test only checks that nothing raises
Phantom dependency	The declaration is deleted but the package is still installed through another dependency	The import resolves at test time, so the build passes
Hide, do not remove	The package is moved to an optional extra or a dev group	CI installs extras, so nothing breaks
Weaken the tests	A failing assertion is relaxed or a test is skipped	The suite is the oracle, and the agent edited the oracle
Trade one for three	The library is replaced by two or three smaller ones	Each installs cleanly; the task said remove X, not reduce dependencies

A change must pass six gates, and each gate rejects one way of cheating



Oracle stack: six gates, each tied to the cheat it rejects.

Gates 1 and 2 are what DI-Bench already does. Gates 3 to 6 are new, and gate 6 is P1's metric. Once the gates exist, the 2026 [hacker-fixer loop](#) is run against them: an attacker agent tries to pass all six without doing the work, and every exploit it finds becomes a fix.

What the paper would claim

Headline: coding agents can remove a used dependency in X percent of cases, but a test-only oracle accepts Y percent of cheats, and a six-gate oracle grounded in the program graph catches Z percent of them.

Three supporting results carry the headline.

1. Refactoring is where dependencies rot, for agents as well as humans. Agent refactoring pull requests leave undeclared imports or dead declarations at K times the human rate. This is the motivation

section, measured on provenance-labelled agent PRs.

2. Execution pass rate is blind to a measurable share of the dependency set. Deleting a ground-truth dependency leaves CI green for N percent of dependencies in DI-Bench, and adding unused packages costs nothing. This is the dependency mutation score, and it is why gates 3 to 6 exist.
3. A deterministic structural signal survives adversarial hardening where rubric judges do not. Unreachable added structure, the same quantity behind P1's anti-bloat term and the dependency pruning in P2, separates genuine from gamed changes on both tasks.

What is new against the 2026 literature: nobody evaluates in-situ dependency removal, nobody has measured manifest-to-import consistency of agent changes, nobody has mutation-tested dependency declarations, and every refactoring benchmark scores quality with an LLM judge. Working title: Remove This Dependency: Refactoring Agents, Dependency Drift, and What Tests Cannot See.

The experiments that decide whether this stands

Five experiments, in the order they unblock each other. Each has a number that confirms the framing and a number that kills it.

#	Experiment	Needs	Confirms if	Kills if
E1	Textual baseline on DI-Bench Python: Qwen-7B patches and pipreqs scored name-only against the oracle	Nothing, runs on this Mac	Static tool is within reach of the 7B model, so the hard part is naming and versioning, not finding imports	Qwen beats pipreqs by a wide margin on recall, so structure is not the bottleneck
E2	Import footprint of the oracle: which ground-truth dependencies are imported, which are reachable from tests, how many have a one or two file footprint	Nothing, static	Over 5% of oracle dependencies are unreachable from tests, and most repos have a narrow-footprint candidate for removal	Under 2% unreachable, so EPR has no blind spot worth a paper, and few removal candidates
E3	Dependency mutation score: delete each oracle dependency, re-run CI, count green runs	Linux box with Docker and sysbox	10% or more of deletions stay green, matching E2's static estimate	Under 3% stay green, so tests already see nearly everything
E4	Removal pilot: 50 DI-Bench Python instances, one narrow-footprint target each, two agents, every output audited by hand against the cheat table	Dataset built from E2, agent API access, the Docker box	Agents claim success often and at least a fifth of claimed successes are a cheat	Agents either fail outright or succeed cleanly, so there is no cheating to catch
E5	Manifest hygiene of agent PRs: provenance-labelled agent pull requests run through FawltYDeps and depcheck before and after, split by whether RefactoringMiner tags the PR as a refactoring	The 2026 PR datasets	Agent refactoring PRs introduce undeclared or dead dependencies at a higher rate than human ones	Agents are no worse than humans, which removes the motivation but not the benchmark

E1 and E2 run today and are reported in the next section. E3 and E4 need the execution harness, which is disabled on Python 3.13 and cannot run on macOS at all, so they need a Linux machine with sysbox. E5 is independent of everything else.

One caveat for E2 and E3: the ground-truth manifests themselves may be bloated. A dependency that is declared, never imported, and deletable without failing CI is a flaw in the benchmark's oracle, not in the model. Report that number separately; it is a result in its own right.

What the first numbers say

E1 and E2 ran today on the 98 Python repositories of DI-Bench regular, with no execution. The framing survives both. One finding about the existing baseline changes what can be claimed from the Kaggle run.

E1, the Qwen-7B baseline. The patches on disk are not what the model produced.

Measure	Value
Patches DI-Bench's evaluator can parse as they stand	58 of 98
Why the other 40 fail	5 empty outputs; 35 where the Kaggle run's output parser took the fenced filename as the file body
Patches recoverable from the raw model output in <code>trajs.json</code>	95 of 98
Name-only precision / recall / F1, patches as they stand	0.29 / 0.15 / 0.20
Name-only precision / recall / F1, recovered output	0.30 / 0.23 / 0.26
Instances where the model declared zero dependencies	21 of 98
Instances with exactly the right set of package names	6 of 98
False positives that are packages never imported anywhere in the repo	217 of 283
Recall on test-reachable dependencies vs the rest	0.24 vs 0.27

The never-imported false positives are mostly dev tools: `pytest-cov`, `coverage`, `ruff`, `black`, `mypy`, `isort`. They all install cleanly, so an execution oracle would never flag them. That is the over-declaration blind spot, observed in a real model's output before constructing anything. Recall is flat across reachable and unreachable dependencies, so the model is weak everywhere rather than weak on the hidden ones.

E1 continued, the static-tool baseline DI-Bench never ran. `pipreqs` reads import statements and maps them to package names with a lookup table. It was run on the same 98 masked repositories, once over every file and once ignoring test, docs and example directories.

Method	Precision	Recall	F1	Exactly right set
Qwen2.5-Coder-7B, recovered output	0.30	0.23	0.26	6 of 98
<code>pipreqs</code> , all files	0.52	0.78	0.62	2 of 98
<code>pipreqs</code> , ignoring tests and docs	0.65	0.78	0.71	14 of 98

A deterministic tool from 2015 recovers more than three times the model's recall and gets the whole set right twice as often. Its recall ceiling of 0.78 is the same ceiling E2 found: the dependencies it misses are the ones no import names, which no import-reading method can find.

This changes two things. Any LLM or agent method on DI-Bench Python has to beat pipreqs, and the paper must report that line. And a code-graph tool whose contribution is finding imports is rediscovering pipreqs; its value has to come from what pipreqs cannot do, which is reachability, the thing that separates a used dependency from a bloated one.

E2, the oracle's import footprint. The blind spot exists and the removal pool is large.

Measure	Value
Ground-truth dependencies across the 98 repos	537
Imported by a mapped name somewhere in the repo	450
Imported and reachable from a test file	405
Imported but reached by no test, the lower bound of what EPR cannot see	45, in 25 repos
Not matched to any import	88
Imported dependencies used by one or two source files	324 of 450
Repos with at least one such narrow-footprint dependency	90 of 98

The 88 unmatched dependencies are a mix, by hand inspection of the 40 with no mention anywhere: import names that differ from package names such as `z3-solver` and `dm-tree`, runtime-only packages such as `ipykernel` and `jaxlib`, dev tools declared as runtime dependencies, redundant transitive listings such as `werkzeug` beside `flask`, and some apparent bloat. So the EPR blind spot is at least 8% of the dependency set and could be a quarter. E3, the mutation score on a Linux box, is what settles it.

The caveat from the experiments table showed up immediately. At least one ground-truth manifest declares `pytest`, `ruff` and `pytest-asyncio` as runtime dependencies, and another declares `type stubs`. Some of DI-Bench's oracle is itself bloated. That is a reportable result, and it means a model penalised for omitting those packages is being penalised for being right.

What this means for the framing: the model is weak and over-declares invisibly, the oracle has a blind spot of measurable size, and nine in ten repositories offer a narrow-footprint dependency to remove. Nothing so far kills it. The next number is E3, and it needs the execution harness on Linux.

Extension: joint deltas of the code graph and the dependency graph

Every change an agent makes moves two graphs at once, and the pair of movements is a signature of what the change actually did. This turns gates 3 to 6 into one deterministic computation and gives each instance a difficulty label.

The two graphs. The code graph has the project's modules, classes and functions as nodes, plus the third-party API symbols they touch, with imports, calls and inheritance as edges. Axon builds it for Python; PyCG and the stitched call graphs of the Drosos study are the research-grade equivalents. The dependency graph has packages as nodes and requires-edges, in two layers: declared, what the manifest says, and resolved, what pip actually installed, readable from the CI log or a freeze.

A change produces a delta in each graph. For a genuine removal of package D the honest delta is specific: every edge from project code into D's API disappears, new project-internal nodes appear carrying the replacement logic and have callees of their own, tests still reach those nodes, D leaves both layers of the dependency graph, and the resolved closure shrinks by D's subtree. Each cheat breaks that pattern in its own way.

Change	Code-graph delta	Dependency-graph delta	Tell
Genuine removal	Edges into D gone; new internal nodes with callees of their own; test-reachability of the rewritten region preserved	D gone from declared and resolved; closure shrinks	Both graphs move together
Vendoring	Large added subgraph structurally similar to D's own call graph; edges now point at the copy	D gone from both layers	Code graph grows by about D's mass
Stubbing	Edges into D gone; added nodes are leaves with no callees	D gone from both layers	Removed structure far exceeds added
Phantom	No change; project code still imports D	D gone from declared, still present in resolved	The two layers disagree
Hide in an extra	No change	D moves from main to optional	Declared layer moves, resolved does not
Weaken tests	Delta confined to test modules; reachability from tests to the former call sites shrinks	None	Test-reachable set shrinks
Trade one for three	Edges into D replaced by edges into D1, D2, D3	D gone, three added; closure may grow	Closure size does not fall

What this adds to the paper.

1. Gates 3 to 6 become graph computations: similarity of the added subgraph to D's own graph for vendoring, triviality of added nodes for stubbing, disagreement between the declared and resolved layers for phantoms, and test-reachability of the rewritten nodes for weakened tests. All execution-free, which is the structural-verifier claim stated concretely.
2. A difficulty label per instance: the used API surface of D in the code graph, meaning the distinct D symbols called, call sites and modules. The footprint-in-files measure used so far is the crude version. Hypothesis: agent success falls and cheating rises with surface size.
3. It unifies the experiments already run. A deletion mutant is the special case where the code graph does not move and D leaves the declared layer; phantom means the resolved layer kept D; a blind spot means D's code edges start only from nodes no test reaches. The graph version makes those file-level numbers symbol-level.
4. It is the same object dependency inference needs. A behaviour-preserving refactoring must leave the external API nodes of the code graph unchanged, which is the metamorphic check from the earlier framing list, so P1 and P2 literally share a data structure.

Precedent, stated honestly. The merged code-plus-dependency graph exists: the Drosos study's fine-grained project dependency graph, built by stitching PyCG call graphs across PyPI, and Eclipse Steady's code-centric reachability from application code to library methods. Call-graph differencing for change impact analysis is twenty years old, Chianti by Ren and colleagues in 2004. Clone detection is mature. What does not exist is the joint delta used as an oracle for agent changes with cheat signatures, and the correlation between the two graphs' movements under dependency co-evolution. That is the claim to make, and only that.

Risks. Python call graphs are incomplete under dynamic dispatch and importlib, and PyCG's precision is good but not perfect, so Axon's call resolution must be validated against PyCG on a sample before any number is trusted. Graph-level clone detection is heavier than text clone detection; start with token-level clones against D's source and move to graph similarity only where text misses rewrites. The resolved layer

needs a freeze per run; the CI logs already carry pip's install lists and a `pip freeze` step can be added to the harness.

#	Experiment	Needs	Confirms if	Kills if
E6a	Signatures on constructed changes: for the 62 strongest instances, generate vendored, stubbed, phantom and genuine variants automatically, with genuine taken from real removal commits where available, compute the joint delta, measure separability	Axon or PyCG, the harness for ground truth	AUC above 0.9 on constructed cheats	Signatures overlap, so the delta cannot separate cheats
E6b	Classify the removal pilot's agent outputs with the signature and compare to the hand audit	E4 outputs	Agreement above 85% with the audit	The classifier disagrees with humans on most cases
E6c	Difficulty: used-API-surface size of D against agent success and cheat rate	E4 outputs	Success falls and cheating rises with surface size	No relation
E6d	Symbol-level re-run of E3: blind spots and phantoms by call-graph reachability instead of file imports	Axon on the 51 gold-passing repos	Symbol-level numbers agree with file-level within a few points	Wide disagreement, meaning the file-level estimate was wrong

E6a and E6d need no agents and no credits, only Axon or PyCG on repositories already on disk, so they started the same day. First results follow.

E6 first results, same day, nothing external. The question was whether the two static graphs can predict, with no execution, which ground-truth dependencies CI will not notice when deleted. The 233 executed deletion mutants on gold-passing instances are the ground truth, 115 of them invisible to CI.

Static predictor of "CI will not notice this deletion"	Predicted	Precision	Recall
Dependency layer: D lies in the requires-closure of the other declared dependencies, from PyPI metadata	73	0.97	0.62
Code layer, file level: no test file reaches a file that imports D	70	0.69	0.42
Both layers combined	118	0.81	0.83

The dependency layer alone is almost never wrong and finds the phantoms. The code layer alone finds the test blind spots but with more noise. Together they recover five in six invisible deletions at four in five precision, from the manifest, PyPI metadata and import statements, with no container and no test run. That is the extension's central claim, confirmed on executed ground truth.

The symbol-level code layer is the weak point, for tooling reasons. PyCG, the research-grade Python call-graph tool, built graphs for 18 of the 51 repos and hung or crashed on 33 even after a bug fix and a four-minute cap. On the 18 it covered, symbol-level "not test-reachable" had precision 0.47 and recall 0.64, worse than the file-level version, because PyCG misses dynamic calls and then reports a dependency as unreachable when it is not. Dependencies with no called symbol at all were invisible in 4 of 5 cases, which is the difficulty label working at the bottom end. Axon runs in under a second per repo but names methods without their class and models no external symbols, so it agrees with PyCG on 99% of test-reachability for shared nodes while matching only a quarter of edges. Conclusion: use the dependency layer plus file-level imports now, and build the symbol-level layer on Axon's internal graph with import-level external edges rather than on PyCG.

E6a result, constructed cheats through the executed harness. For the 62 strongest removal instances, three cheats were generated automatically and run through the real CI: hide, moving the dependency into an optional extra or the dev group; vendor, copying the library's own wheel source into the repository; stub, a local package shadowing the library whose imported symbols return nothing. 176 variants ran in 10 shards.

Cheat	Variants	Pass CI, gate 2 alone	Caught by another gate when CI passes	Which gate
Hide in an extra	62	35	35 of 35	Declaration still present, or package installed anyway
Vendor the source	52	18	18 of 18	Clone similarity to the library's source, 51 of 52 vendor patches flagged
Stub the symbols	62	8	0 of 8	None of the install-level gates

Three readings. First, CI alone accepts a cheat in a third of cases overall, and more than half the time for the simplest one, moving the declaration to an extra that CI installs anyway. Second, the install-level gates close hide and vendor completely, which validates gates 1, 3 and 5 on executed ground truth. Third, stubbing is the real adversary: the 8 stub variants that pass CI are invisible to every install-level check because nothing is installed and nothing is copied; their replacement code is a handful of functions that return nothing, which is precisely the structural signature, added nodes with no callees, that gate 4 and the code-graph delta exist to flag. The removal benchmark's oracle therefore needs the structural layer, not only the install layer, and the paper can now say so with numbers instead of an argument.

Glossary

Term	Meaning here
Manifest	The file that declares a project's dependencies: <code>pyproject.toml</code> , <code>requirements.txt</code> , <code>package.json</code>
Dependency inference	Given the code and a manifest with its dependencies removed, write them back. DI-Bench's task
EPR, execution pass rate	DI-Bench's score: apply the model's manifest, build in a container, run CI. Pass or fail
Name-only precision and recall	DI-Bench's textual score. Precision: share of predicted packages that are correct. Recall: share of true packages that were predicted. Versions ignored
Oracle	The source of the true answer. The ground-truth manifest, or the CI run
Bloated dependency	Declared in the manifest but never used by the code
Phantom dependency	Used by the code but not declared. It works only because another package pulled it in
Test-reachable	A dependency imported by a file that some test imports, directly or through other project files. Only these can affect EPR
Dependency mutation score	Delete each true dependency one at a time and re-run CI. The share of deletions that still pass. High means tests are blind
Footprint	How many source files import a package. A narrow footprint makes removal tractable
Vendoring	Copying a library's source into your own repo instead of depending on it

Term	Meaning here
Stubbing	Replacing a real call with a function that returns a fixed value or does nothing
Behaviour-preserving	The program does exactly what it did before. The defining property of a refactoring
Metamorphic relation	A transformation whose effect on the answer is known in advance. A rename refactoring must not change the dependency set
Reward hacking, gaming	Making the score go up without doing the task
Hacker-fixer loop	An attacker agent tries to pass a verifier without solving the task, a fixer patches the verifier, a solver checks real solutions still pass
RefactoringMiner	A tool that detects which refactorings a commit performed. Supports Java, Python, C++
pipreqs, FawltYDeps, deptry	Static tools that read imports and compare them to the manifest
Axon	The code knowledge-graph tool in this repo. Builds import, call and reachability graphs for Python, JavaScript, TypeScript

Sources

- [DI-Bench, Microsoft, 2025](#)
- [Bloat beneath Python's Scales, Drosos et al., FSE 2024](#)
- [An Empirical Study of Build Maintenance Effort, McIntosh et al., ICSE 2011](#)
- [Can We Trust Tests To Automate Dependency Updates?, Hejderup and Gousios](#)
- [TimeMachine-bench, EACL 2026](#)
- [SWE Atlas, 2026](#)
- [Understanding Security Risks of AI Agents' Dependency Updates, 2026](#)
- [A Large-Scale Empirical Study on Java Library Migrations, He et al.](#)
- [Hardening Agent Benchmarks with Adversarial Hacker-Fixer Loops, 2026](#)
- [PyTrim, 2025](#)
- [MigrationBench, 2025](#)
- [PyMigBench](#)
- [pipreqs, FawltYDeps](#)
- Numbers in this doc: scripts `analyze_python.py` and `score_pipreqs.py` in the session scratchpad, run on DI-Bench regular Python, 98 instances, 1 October 2026

What the executed numbers say

The DI-Bench harness now runs on free GitHub Actions runners under the sysbox runtime, with the evaluator's logic unchanged. Gold patches, the recovered Qwen patches, and 531 single-dependency deletion mutants were all executed on the Python regular set within a day. Two upstream harness bugs were found and fixed on the way: the fake-library check crashed on git-URL dependencies, and act's "Job succeeded" was trusted even when the job never started, which produced two false gold passes. Those two instances are excluded below.

Measure	Value
Gold patches that still pass CI, 21 months after release	51 of 96
Gold failures at install time: Poetry's installer dropped old Pythons, a discontinued Mambaforge download, Python 3.8 no longer fetchable	about 17
Gold failures in the project's own tests or linters, from unpinned libraries moving	about 23
Qwen-7B execution pass rate, all 95 instances	6 of 95
Qwen-7B execution pass rate, gold-passing instances only	5 of 50, 10%
DI-Bench's reported figure for the same model and prompt, January 2025	22.4%

Half of DI-Bench's Python oracle has rotted. That is the drift this paper is about showing up inside the benchmark itself, and it means every execution number must be conditioned on gold-passing instances. The conditioned Qwen figure is in the range DI-Bench reported, so the environment is faithful enough to trust.

E3, the dependency mutation score. Each mutant is the oracle manifest with one dependency deleted, scored on instances whose gold patch still passes.

Measure	Final value
Mutants of gold-passing instances	233
Deletions that still pass CI	115, 49%
of which phantom: the package was installed anyway through another dependency or a tox list	91
of which blind spot: the package was absent and the tests still passed	24, 10.3% of mutants, in 15 of 51 instances
Instances with at least one deletion CI cannot see	38 of 51
Sanity check: visible deletions where the package was indeed absent	114 of 118

Two effects, and the split matters. The large one is phantom redundancy: half of DI-Bench's ground-truth declarations can be deleted with no effect because the package arrives transitively, which is the Drosos study's second-largest bloat category sitting inside the oracle. DI-Bench's curation only verified that deleting all dependencies fails, never one. The smaller one is the true test blind spot, a tenth of ground-truth dependencies, at the confirm line in the experiments table. Both are invisible to execution pass rate, and both are exactly what gates 4 and 5 of the removal oracle exist to catch. The framing stands on all three measured experiments; what remains is the removal pilot and the agent-PR study.